Sorting

Bubble sort

Quicksort on hypercube

Quicksort on PRAM

Merge sort

Sorting networks

© 2009-2021 Vivek Sarin

Sorting Algorithms

Algorithm	Worst Case Complexity	Average Case Complexity
Bubble Sort	n^2	n^2
Insertion Sort	n^2	n^2
Merge Sort	n log n	n log n
Quick Sort	n^2	n log n
Heap Sort	n log n	n log n

© 2009-2021 Vivek Sarin

Bubble Sort

Serial Algorithm

For k = 1, … , n (n=number of elements)

For i = 1, … , n-k,

compare-exchange (a_i, a_{i+1}) [exchange only if $a_i > a_{i+1}$]

Parallel Bubble Sort

- Odd-even transposition

  For k = 1, … , n

  - Odd exchange step: compare-exchange (a_i, a_{i+1}) where i is odd
  - Even exchange step: compare-exchange (a_i, a_{i+1}) where i is even
- Use n/2 processors to do each exchange step

- Exercise: Show that the algorithm works correctly
  - Hint: after k phases, an element is within n-k positions of its final location in the sorted list
- Exercise: Implement parallel bubble sort on a 1D torus

© 2009-2021 Vivek Sarin

Parallel Bubble Sort

- Use n/2 processors
  - Parallel complexity
    - T_p = n phases, 2 steps per phase
    - T_p = O(n) using p = n/2 processors
  - Processor-time product: $O(n^2)$
- Use p processors
  - p phases of compare-exchange of local sorted lists
  - Parallel complexity

$$T_p = \left(\frac{n}{p}\log\frac{n}{p}\right) + p\cdot\frac{n}{p} + \frac{n}{p}\cdot p$$

Local sort Phases Communication

  - Use p = log n to get T_p = O(n)
  - Processor-time product: O(n log n)

© 2009-2021 Vivek Sarin

Quick Sort – Serial Algorithm

Quicksort(A[1,…,n])

Choose pivot, say A[1]

Reorder elements of A such that

A[1,…,m-1] < pivot and

A[m+1,…,n] > pivot

Set A[m] = pivot

Quicksort(A[1,…,m])

Quicksort(A[m+1,…,n])

© 2009-2021 Vivek Sarin

Quick Sort – Parallel Algorithm

- Hypercube with $p = 2^d$ processors
- Each processor has n/p elements that have been sorted using a serial algorithm such as quick sort
- Algorithm
  1. Choose pivot, broadcast to all processors
  2. Each processor partitions its list into two sublists: one with values larger than pivot, and one with values smaller than pivot
  3. Each processor exchanges appropriate sublist with neighbor along dimension d (higher id processor gets the two sublists with values larger than the pivot)
  4. Split hypercube into two (d-1) dimension hypercubes
  5. Repeat 1-4 recursively for the two (d-1) hypercubes

© 2009-2021 Vivek Sarin

Quick Sort – Parallel Algorithm …

- Parallel complexity: assume perfect pivots

$$T_p = \left(\frac{n}{p}\log\frac{n}{p}\right) + \log p\left[\log p + \log\frac{n}{p} + \frac{n}{p} + \frac{n}{p}\right]$$

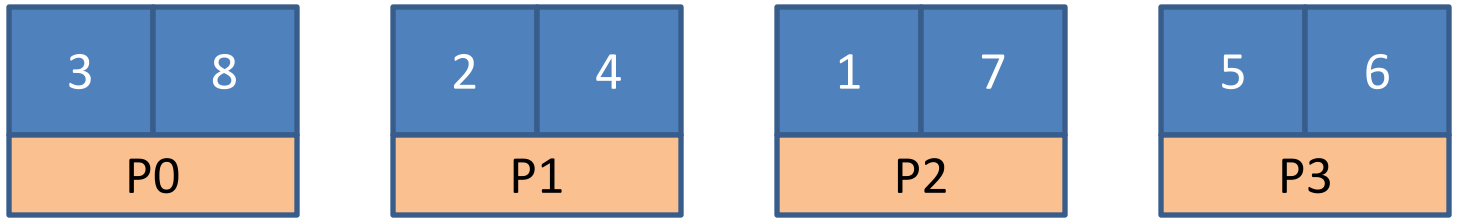
Local sort — Steps — Broadcast Pivot — Split Array — Merge Array — Communication

- Pivot selection
  - Bad pivot causes bad performance
  - Choose pivot randomly
  - Choose pivot randomly from median of processor lists
  - Choose mean of medians

© 2009-2021 Vivek Sarin

Parallel Quick Sort Example

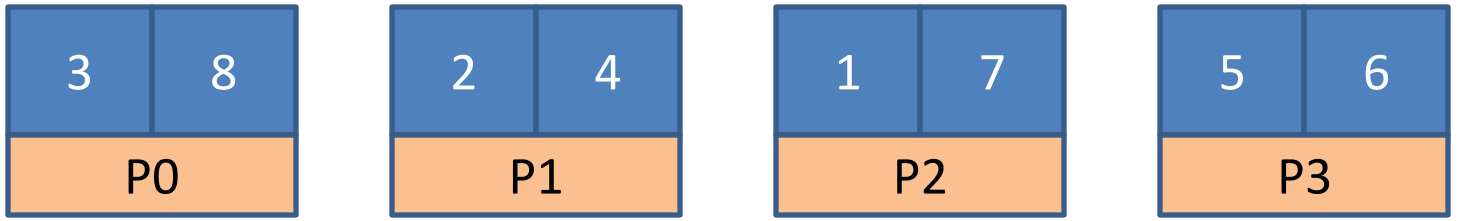
After sorting local lists, we have:



© 2009-2021 Vivek Sarin

Parallel Quicksort Example

After sorting local lists, we have:



First Step

Medians of lists on the processors are: 3, 2, 1, 5

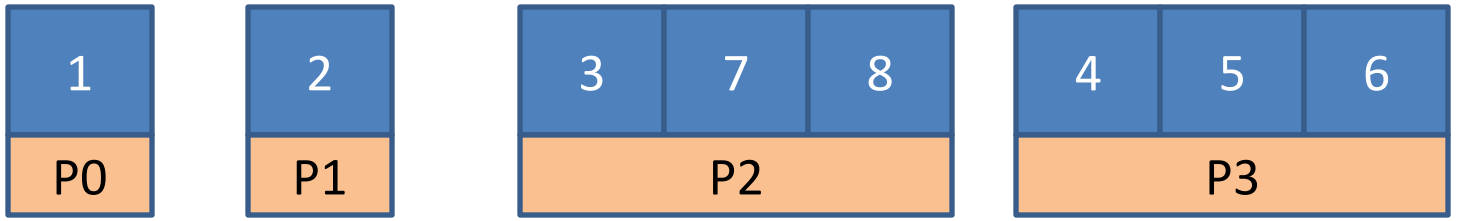
⇒Mean of medians is (3+2+1+5)/4=11/4=2.75

⇒Pivot = 2.75

Each processor splits its list at the pivot, i.e., at 2.75

P0 and P2 exchange sublists; P1 and P3 exchange sublists

Each processor merges its sublists to get sorted local list



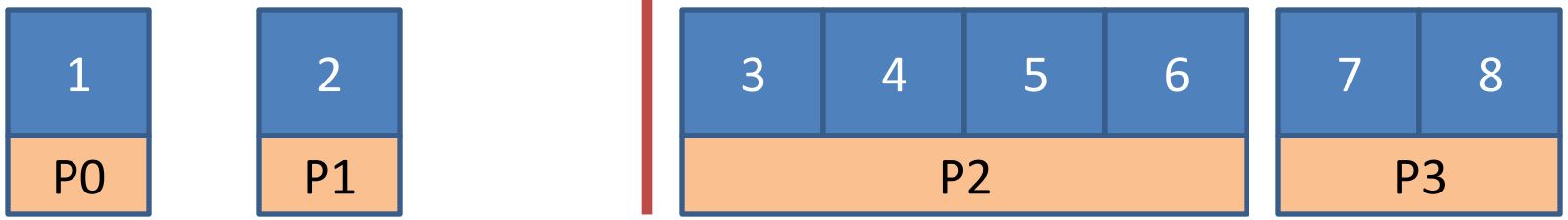
© 2009-2021 Vivek Sarin

Parallel Quicksort Example …

Second Step

Median of lists on P0 &P1: 1, 2
Mean of medians is (1+2)/2=1.5
⇒Pivot for P0 & P1 = 1.5
P0 and P1 split their lists at the pivot, i.e., 1.5
P0 and P1 exchange sublists (NO CHANGE)

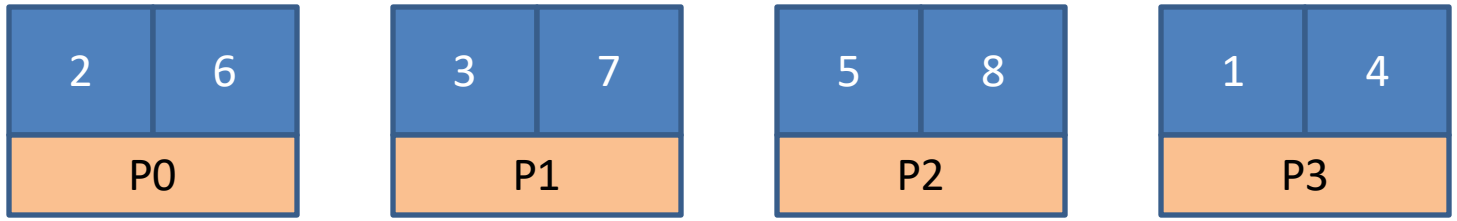
Median of lists on P2 & P3: 7, 5
⇒Mean of medians is (7+5)/2=6
⇒Pivot for P2 & P3 = 6
P2 and P3 split their lists at the pivot
P2 and P3 exchange sublists
Each processor merges its sublists to get sorted local list



At this point, the algorithm terminates.
The sorted list can be obtained by concatenating the lists on P0, P1, P2, and P3 (in that order)

© 2009-2021 Vivek Sarin

Exercise



© 2009-2021 Vivek Sarin

PRAM: Parallel Random Access Memory

- Concurrent memory accesses (reads and writes) cause conflicts that must be resolved
- EREW – exclusive read exclusive write
  - a memory location can be read or written by a single processor at any time
- CREW – concurrent read, exclusive write
  - a memory location can be read by multiple processors at the same time, but can be written by a single processor only; concurrent write requests are serialized
- CRCW – concurrent read, concurrent write
  - a memory location can be read or written by multiple processors at the same time
  - write conflicts resolved by various policies – highest priority processor wins, arbitrary processor wins, etc.
- ERCW – exclusive read, concurrent write
  - Not interesting

Exercise: Implement Bubble Sort on CRCW PRAM

© 2009-2021 Vivek Sarin

Quick Sort

- CRCW PRAM algorithm
- Construct binary tree to index elements, do not rearrange elements
- Use n processors

Parallel Quick Sort (PRAM)

1. Choose pivot (all processors write element in the root)
2. All processors read pivot
3. Processor decide which subtree they belong to (left: values lower than pivot, right: values higher than pivot)
4. Create left and right child of root
5. Repeat 1-4 until done

- Parallel complexity: O(log n)
- Exercise: From the tree created by parallel quick sort algorithm (above) construct a sorted list of elements in the shortest time possible using n processors

© 2009-2021 Vivek Sarin

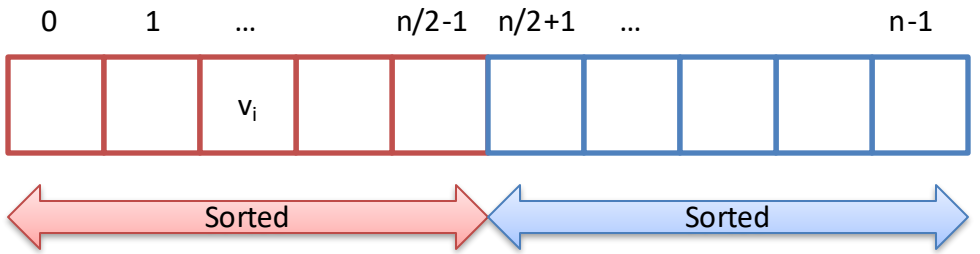
Merge Sort

Merge Sort

1. Recursively sort two halves of the list
2. Merge the sorted halves

Parallel Merge Sort

- Consider n processors are available to sort n values
- Suppose the two halves of the list are sorted



- Processor i determines how many values are smaller than v_i by doing a binary search on the right half in O(logn) steps

$$T(n) = T\left(\frac{n}{2}\right) + \log n = (\log n)^2$$

- Parallel Complexity: $(\log n)^2$, Processor-time product: $n(\log n)^2$

© 2009-2021 Vivek Sarin

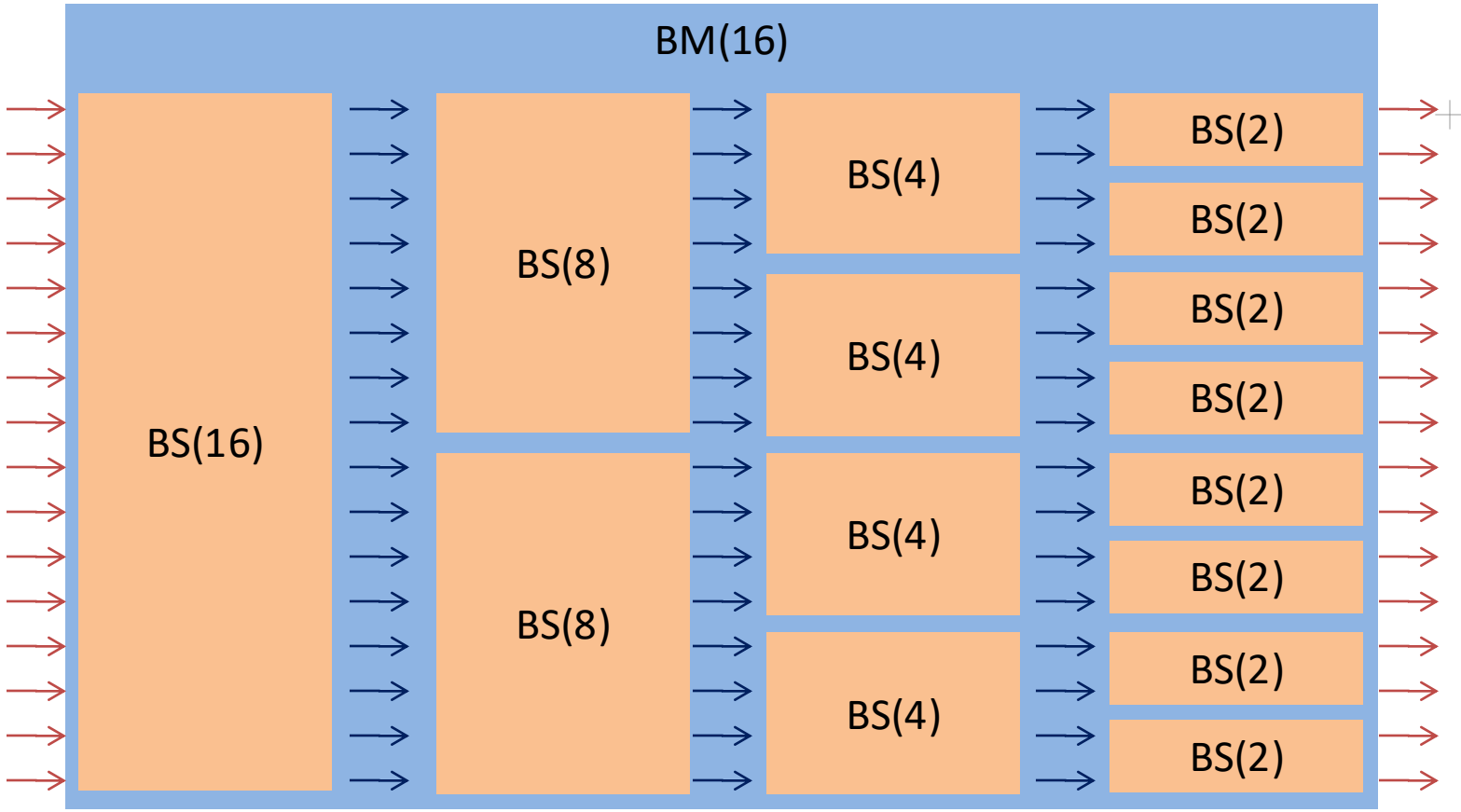
Sorting Networks

- Sort n elements in $(\log n)^2$ time
- Bitonic sequence: a sequence of elements $a_0, a_1, \ldots, a_{n-1}$ s.t. for some i
  - $a_0, \ldots, a_i$ is monotonically increasing
  - $a_{i+1}, \ldots, a_{n-1}$ is monotonically decreasing
  - or the sequence can be made so after cyclic shift

- Bitonic Split: compare-exchange a_i and $a_{i+n/2}$ for i=0,1,…,n/2-1
  - A bitonic split of a bitonic sequence of length n divides the elements into two bitonic sequences of length n/2 each, where the elements of the first sequence are smaller than the elements of the second sequence

© 2009-2021 Vivek Sarin

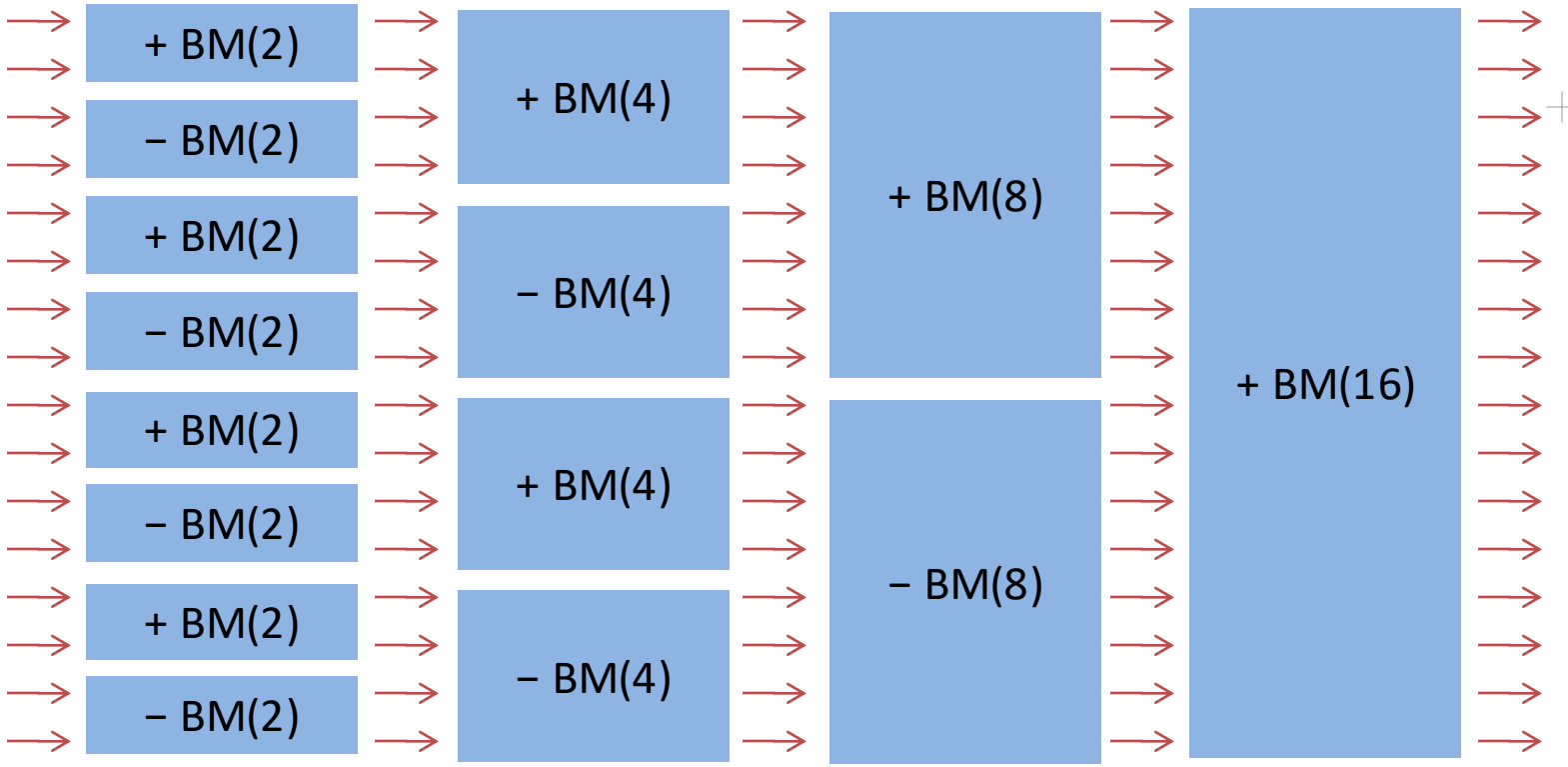
Sorting a Bitonic Sequence



BS(x) Bitonic split module for x elements
BM(x) Bitonic sorting module

© 2009-2021 Vivek Sarin

Sorting Network Example



+ BM(x) Bitonic sorting module for x elements (ascending order)
− BM(x) Bitonic sorting module for x elements (descending order)

© 2009-2021 Vivek Sarin